RESEARCH PAPER REVIEW AND IMPLEMENTATION REPORT

ON THE ARTICLE

MASTERING THE GAME OF GO WITHOUT HUMAN KNOWLEDGE

Submitted By

Shaik Shafreed (2024BCS0113)

Chandrakanth (2024BCS0125)

Arya Vardhan (2024BCS0129)

Shaik Humair Fatah (2024BCS0105)

Lakavath Harshithan (2024BCS0321)

Course: CSE 311 Artificial Intelligence

Course instructor: Dr. A V Prajeesh

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM



Contents

Page No.

1. Introduction.....1

2. Motivation.....2

3. Problem Statement.....3

4. Literature Survey.....4

5. Review of the Proposed Methodology.....6

6. Results and Discussion.....12

7. Conclusion.....16

8. Future Work.....17

9. References.....18

10. GitHub Repository and Reproducibility Details.....19

1. Introduction

Go is a two-player board game in which players alternately place black and white stones on the intersections of a grid, capturing opponent stones by surrounding them and competing to control the larger area. Its rules are short, but the number of legal positions on the standard 19×19 board is about 2×10^{170} [9], and good play depends on long-range judgement that is hard to express as hand-written rules. For decades Go was regarded as a grand challenge for artificial intelligence.

AlphaGo [2] was the first program to defeat a professional player on a full board. It combined Monte Carlo tree search (MCTS) with deep neural networks that were first trained on positions from human expert games. The article reviewed in this report, *Mastering the game of Go without human knowledge* by Silver et al. [1], introduced **AlphaGo Zero**. It removes human data entirely: a single neural network, starting from random weights, is trained only on games the program plays against itself, using MCTS as a policy-improvement operator. After three days of training it beat the version of AlphaGo that defeated Lee Sedol by 100 games to 0 [1].

In this project we studied the paper and implemented the complete AlphaGo Zero training loop in Python and PyTorch [10] on a 12×12 board, scaled down to run on a single laptop. Our contributions are:

- A Go environment with captures, superko, passing and area scoring.
- A dual-head residual network that outputs a move distribution and a position value.
- MCTS with the PUCT selection rule, Dirichlet root noise and temperature-based move selection.
- A batched, resumable self-play pipeline that runs 18 games in parallel on the CPU.
- Training on the GPU with the AlphaGo Zero loss, and an arena that promotes a candidate only if it wins at least 55% of its games against the current best model.
- A four-stage curriculum run for eight candidate iterations (3–10), with full results.

Sections 2–4 give the motivation, problem statement and literature survey. Section 5 reviews the paper's method alongside our implementation, Section 6 presents the results, and Section 10 explains how to reproduce them.

2. Motivation

AlphaGo Zero is one of the clearest demonstrations that a system can reach superhuman performance in a complex domain **without human examples**. Its predecessor depended on 30 million positions from expert games [2]; AlphaGo Zero needs only the rules. This matters beyond Go: human data is expensive, may be unavailable, and can cap a system at human-level ideas. The same algorithm, generalised as AlphaZero, went on to master chess and shogi [3].

The paper is also a good subject for an artificial-intelligence course because it combines three core topics in one system:

- **Search:** MCTS looks ahead to choose moves, balancing exploration and exploitation.
- **Learning:** a deep residual network learns to evaluate positions and propose moves.
- **Reinforcement learning:** self-play produces the training data, and the reward is only the final result of each game.

Finally, the method is simpler than AlphaGo's: one network instead of separate policy and value networks, and no random rollouts. That makes it feasible to implement from scratch. By running it at a small scale we could observe learning directly, and verify claims of the paper with our own experiments rather than only reading about them. As Section 5.12 shows, this also exposed subtle implementation errors that a paper summary would not reveal.

3. Problem Statement

Goal. Given only the rules of Go on a 12×12 board, learn a function $f_\theta(s) = (p, v)$, where p is a probability distribution over the 145 possible actions (144 intersections and pass) and $v \in [-1, 1]$ predicts the final result for the player to move, such that MCTS guided by f_θ plays increasingly strong Go. The only training signal is the outcome of games the system plays against itself.

Inputs and outputs. A position s is encoded as three 12×12 planes: the current player's stones, the opponent's stones, and the empty points. The network returns the policy p and value v. MCTS returns an improved policy π, given by the visit counts at the root, which is used both to choose moves and as a training target.

Objective. Minimise the AlphaGo Zero loss: the cross-entropy between π and p, plus the squared error between the game result z and v, plus L2 regularisation of the weights (Section 5.8).

Success criterion. A newly trained candidate network replaces the current best network only if it wins at least 55% of an evaluation match against it. Progress is therefore measured by promotions and by the training loss.

Constraints. All training ran on one laptop with a 20-thread CPU, 15 GB of RAM and an NVIDIA RTX 4050 Laptop GPU with 6 GB of memory. This limits the network size, the number of search simulations per move and the number of self-play games, compared with the paper's large-scale setup. Komi and resignation are not used.

4. Literature Survey

4.1 Monte Carlo tree search

Classical game programs search a game tree with minimax and a hand-crafted evaluation function. This works poorly in Go, where the branching factor is large and good evaluation functions are hard to write. Coulom [4] introduced *Monte Carlo tree search*, which grows a search tree selectively and evaluates leaves by playing random games to the end. Kocsis and Szepesvári [5] proposed UCT, which treats the choice of move at each node as a multi-armed bandit and applies the UCB1 rule, giving a principled balance between exploring new moves and exploiting good ones. MCTS programs such as Crazy Stone and MoGo became the strongest Go programs of the following decade.

4.2 Prior-guided search: PUCT

Rosin [6] extended bandit-based search with *predictor* probabilities that bias exploration towards moves a predictor considers likely. AlphaGo and AlphaGo Zero use a variant of this rule, PUCT, in which the neural network's policy output provides the prior for every move (Section 5.5).

4.3 Deep residual networks

He et al. [7] showed that very deep convolutional networks can be trained reliably if each block learns a residual that is added back to its input through a skip connection. Combined with batch normalisation [8], residual blocks allow networks of dozens of layers to train stably. AlphaGo Zero replaced AlphaGo's plain convolutional networks with a residual tower and reported a large gain in playing strength from this change alone [1].

4.4 AlphaGo

AlphaGo [2] trained a policy network by supervised learning on 30 million positions from human games, improved it by policy-gradient reinforcement learning through self-play, and trained a separate value network to predict game outcomes. During play, MCTS combined the policy prior, the value network and fast random rollouts. AlphaGo defeated the European champion Fan Hui by five games to nil and, in 2016, world champion Lee Sedol by four games to one.

4.5 AlphaGo Zero

AlphaGo Zero [1], the subject of this review, makes four changes: it learns from self-play only, with no human data; it uses only the black and white stones as input features; it uses a single network with policy and value heads; and it evaluates leaves with the value head alone, without rollouts. MCTS acts as a policy-improvement operator, and the network is trained to match the search's output. Section 5.1 summarises the method in detail.

4.6 AlphaZero

AlphaZero [3] applied the same algorithm to chess, shogi and Go with a single set of hyper-parameters. It dropped the evaluation step: one network is updated continually, and self-play always uses the latest weights. It defeated the strongest existing programs in all three games.

Table 1: Comparison of the AlphaGo family and this project

	AlphaGo [2]	AlphaGo Zero [1]	AlphaZero [3]	This project
Human data	30M expert positions	None	None	None
Networks	Policy + value (separate)	One dual-head ResNet	One dual-head ResNet	One dual-head ResNet
Leaf evaluation	Value net + rollouts	Value head only	Value head only	Value head only
Input features	48 hand-crafted planes	17 stone-history planes	Game-specific planes	3 planes
Model selection	—	Gated, 55% of 400 games	Continuous update	Gated, 55% of 20–40 games
Board	19×19 Go	19×19 Go	Chess, shogi, Go	12×12 Go

5. Review of the Proposed Methodology

5.1 Method of the paper

AlphaGo Zero's training pipeline has three components that run asynchronously in parallel [1]:

1. **Self-play.** The current best network plays games against itself. Every move is chosen by MCTS with 1,600 simulations, and each position is stored with the search probabilities π and, at the end of the game, the winner z.
2. **Optimisation.** The network is trained with stochastic gradient descent on mini-batches sampled from the most recent 500,000 self-play games, minimising $l = (z - v)^2 - \pi^T \log p + c\|\theta\|^2$.
3. **Evaluation.** Each new checkpoint plays 400 games against the current best network and replaces it if it wins more than 55% of them.

The network takes 17 binary planes as input: the stones of each player over the last eight positions and a plane for the colour to play. It uses a residual tower of 20 or 40 blocks with 256 filters. Exploration comes from Dirichlet noise added to the root priors ($\alpha = 0.03$, $\varepsilon = 0.25$) and from sampling moves in proportion to visit counts for the first 30 moves of each game. The 20-block version was trained for three days on 4.9 million self-play games and beat AlphaGo Lee 100–0. The 40-block version, trained for 40 days, beat AlphaGo Master 89–11 [1].

5.2 Overview of our implementation

Our implementation follows the same loop, run as discrete iterations rather than asynchronously (Figure 1). Each iteration plays a batch of self-play games with the current best network, trains a candidate network on the new and recent data, and evaluates the candidate against the best network. The trainer, `train_batched.py`, saves self-play games in batches of 25, so an interrupted run resumes from the last completed batch.

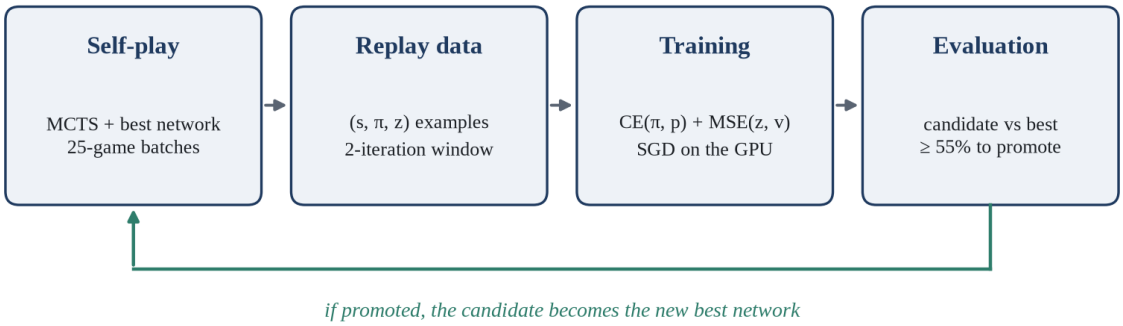


Figure 1: *Training loop of our implementation*

5.3 Go environment

The board (`environment/go_game.py`) is a 12×12 array with 0 for empty, 1 for white and −1 for black. The environment implements the following rules:

- **Captures:** after a move, opponent groups with no liberties are removed.
- **Suicide:** a move that leaves its own group without liberties is illegal unless it captures.
- **Ko:** positional superko. A move may not recreate any earlier board position, which also covers simple ko.
- **Passing:** pass is always legal, and consecutive passes end the game.
- **Scoring:** area scoring, counting stones plus empty regions surrounded by one colour, with no komi. Games longer than 500 moves are stopped and scored.

For the network, positions are encoded from the point of view of the player to move, as three planes: own stones, opponent stones and empty points. Using the mover's perspective means the same network serves both colours.

5.4 Dual-head residual network

`GoNetwork` (`network/network.py`) follows the paper's architecture at a smaller size (Table 2). It has 793,144 trainable parameters, about 3.2 MB per saved model.

Table 2: Architecture of GoNetwork for a 12×12 board

Stage	Layers	Output
Input	Own stones, opponent stones, empty points	3 × 12 × 12
Stem	3×3 convolution, 64 filters, batch norm, ReLU	64 × 12 × 12

Stage	Layers	Output
Residual tower	10 blocks; each: two 3×3 convolutions with batch norm, skip connection, ReLU	64 × 12 × 12
Policy head	1×1 convolution (2 filters), batch norm, ReLU, fully connected 288 → 145, softmax	p: 145 probabilities
Value head	1×1 convolution (1 filter), batch norm, ReLU, fully connected 144 → 64, ReLU, 64 → 1, tanh	v ∈ [−1, 1]

5.5 Monte Carlo tree search with PUCT

Each node of the search tree stores, for every move a, a prior P(s,a) from the network, a visit count N(s,a), and the mean value Q(s,a) of the simulations that passed through it. One simulation has three steps:

1. Select. Starting at the root, repeatedly choose the child that maximises Q(s,a) + U(s,a), where

$$U(s,a) = c_puct \cdot P(s,a) \cdot \sqrt{N(s)} / (1 + N(s,a)), \quad c_puct = 1.5$$

N(s) is the parent's visit count. The U term is large for moves with a high prior that have been visited little, so search first follows the network's suggestions and later relies on measured values Q.

2. Expand and evaluate. At a leaf, run the network once to obtain priors for its children and a value v for the position. No random rollout is played.

3. Back up. Add v to every node on the path back to the root, flipping the sign at each level, so each node holds the value from the point of view of the player who moved into it.

After all simulations for a move, the search policy is $\pi(a) \propto N(s,a)^{(1/\tau)}$, using the visit counts at the root. The number of simulations per move grows with the curriculum, from 35 to 100 (Section 5.10).

5.6 Exploration

Self-play needs variety, or the same games repeat and teach the network nothing new. Two mechanisms from the paper provide it:

- **Dirichlet noise at the root:** $P'(s,a) = (1 - \varepsilon) \cdot P(s,a) + \varepsilon \cdot \eta_a$ with $\eta \sim \mathrm{Dir}(\alpha)$ and $\varepsilon = 0.25$. We scale α with the number of moves, $\alpha = 10 / 144 \approx 0.069$. For a 19×19 board the same rule gives the paper's $\alpha \approx 0.03$.

- **Temperature:** for the first 30 moves of each self-play game, moves are sampled in proportion to visit counts ($\tau = 1$). After that, the most-visited move is played ($\tau \to 0$).

5.7 Self-play and the replay window

Self-play runs in 18 worker processes, each with its own random seed, and games are saved in batches of 25. Every position of every game becomes one training example (s, π, z): the encoded board, the MCTS visit distribution and the final result from the mover's point of view.

Instead of the paper's buffer of the last 500,000 games, each iteration trains on its own games plus those of the previous iteration (a replay window of two iterations). At stage 5 this gave about 416,000–446,000 training positions per iteration.

5.8 Training

The network is trained to predict the search policy and the game result with the AlphaGo Zero loss:

$$L(\theta) = -\Sigma_a\, \pi(a)\, \log p(a) \;+\; (z - v)^2 \;+\; c\|\theta\|^2$$

The L2 term is applied as weight decay, $c = 10^{-4}$. We use SGD with momentum 0.9, a learning rate of 0.001 and mini-batches of 32 positions, for 4 to 8 epochs per iteration depending on the stage. Each candidate starts from the weights of the current best model, and a checkpoint is saved after every epoch. Because `GoNetwork` already outputs probabilities, the trainer passes their logarithm to the loss. Applying a second softmax would distort the policy gradient (Section 5.12).

5.9 Evaluation and promotion

A candidate plays the current best model in an arena match. Colours alternate between games. The first 8 moves of each game are sampled from the visit counts so that games differ, and games are capped at 500 moves and scored by area. Iterations 3–7 used 20 games; from iteration 8 we raised this to 40 games. The candidate is promoted if it wins at least 55%.

5.10 Training curriculum

To use limited compute well, training runs through stages of increasing difficulty (Table 3). Early stages play many short, cheap games with shallow search; later stages search deeper, which gives stronger training targets once the network is good enough to benefit from them. Scripts for an

easier stage 1 (20 simulations per move) also exist, but the candidates reported here start at stage 2.

Table 3: Curriculum stages

Stage	Level	Games per iteration	Simulations per move	Epochs	Iterations
2	Intermediate	400	35	4	3–4
3	Difficult	600	50	5	5–6
4	Expert	850	80	6	7–8
5	Super expert	1,000	100	8	9–10

5.11 Differences from the paper

Table 4 lists the main differences. Most are reductions in scale forced by running on one laptop instead of a large cluster; the algorithm itself is unchanged.

Table 4: Paper settings compared with our implementation

Aspect	AlphaGo Zero [1]	This project
Board	19×19	12×12
Input planes	17 (8-step history + colour to play)	3 (own, opponent, empty)
Residual tower	20 or 40 blocks, 256 filters	10 blocks, 64 filters
Simulations per move	1,600	35–100
Self-play games	4.9 million (3-day run)	5,700 (iterations 3–10)
Training data	Last 500,000 games	Last 2 iterations
Mini-batch	2,048	32
Evaluation	400 games, 55%	20–40 games, 55%
Pipeline	Asynchronous	Sequential iterations
Rules	Chinese rules, komi 7.5, resignation	Area scoring, no komi, no resignation

5.12 Implementation issues found and fixed

Reproducing the method exposed several errors in our first implementation. Each one silently degraded learning without causing a crash, which is why they are worth recording:

- **MCTS backup sign.** Nodes stored values from the point of view of the player to move at that node, while selection maximised the child's value. The search therefore chose the moves that were best for the opponent. With near-random value heads the error was invisible, but

once a model learned an accurate value head, a candidate lost 0–20. After the fix the same model scored 9–11, and a regression test now covers it.

- **Input mismatch.** Training data used an empty-points third plane, while the search evaluator fed a constant black-to-move plane. Every model was trained on one input format and played with another.
- **Double softmax.** The network already outputs probabilities, and the loss applied a second softmax, which weakened the policy gradient.
- **Deterministic evaluation.** With no randomness, 20 evaluation games were really 2 games repeated, and they were cut off at 200 moves. Opening sampling and the 500-move cap fixed this.
- **Silent fallback and missing noise.** Model loading fell back to random weights on failure, and Dirichlet noise was not actually applied in self-play. Both were fixed.

All self-play data generated before these fixes was discarded and regenerated, so every result in Section 6 was produced by the corrected code.

6. Results and Discussion

6.1 Experimental setup

Training ran from 6 to 10 October 2026 on a laptop with a 20-thread CPU (18 self-play workers), 15 GB of RAM and an NVIDIA GeForce RTX 4050 Laptop GPU (6 GB), using Python 3.14, PyTorch 2.11 and CUDA 12.8. Iterations 3–9 ran on Windows; iteration 10 ran mostly on Linux. The code base has 99 passing unit tests covering the environment, network shapes, MCTS, self-play, training and evaluation; two further web-server tests require the `httpx` package.

6.2 Evaluation results

Table 5 and Figure 2 show the result of every candidate. Three candidates were promoted: iterations 4, 7 and 9. The final best model is **iteration 9**, which beat iteration 7 by 29 games to 11 (72.5%).

Table 5: Evaluation of each candidate against the current best model

Iteration	Stage	Opponent	Games	Wins–losses–draws	Win rate	Outcome
3	2	Pre-curriculum best	20	6–14–0	30%	Rejected
4	2	Pre-curriculum best	20	14–6–0	70%	**Promoted**
5	3	Iteration 4	20	9–10–1	45%	Rejected
6	3	Iteration 4	20	10–10–0	50%	Rejected
7	4	Iteration 4	20	11–9–0	55%	**Promoted**
8	4	Iteration 7	40	21–18–1	52.5%	Rejected
9	5	Iteration 7	40	29–11–0	72.5%	**Promoted**
10	5	Iteration 9	40	20–20–0	50%	Rejected

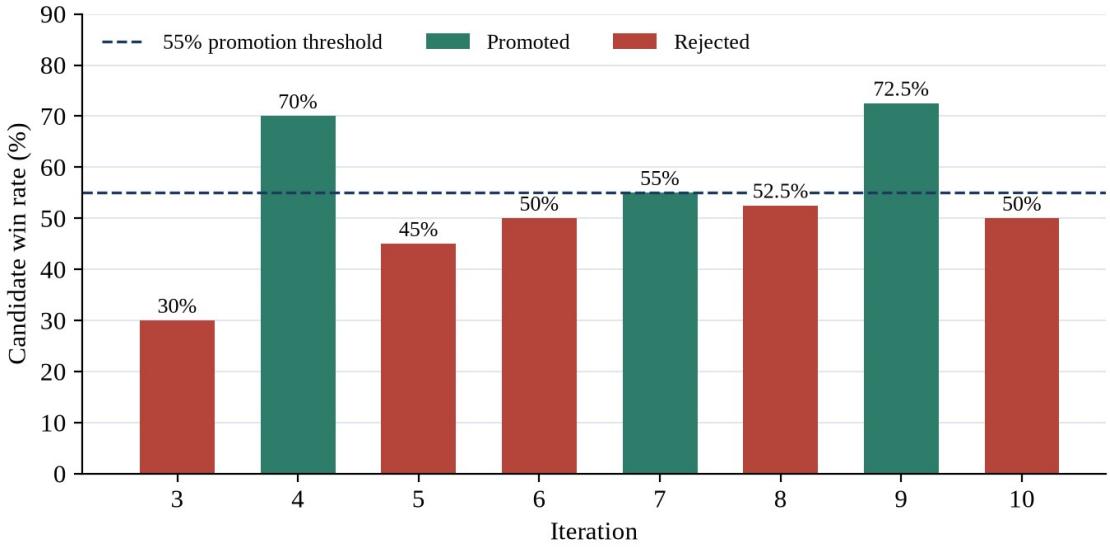


Figure 2: *Win rate of each candidate; the dashed line is the promotion threshold*

6.3 Training loss

Table 6 and Figure 3 show the loss at the end of training in each iteration. The policy loss fell from 4.78 to about 3.25. For comparison, a uniform guess over 145 moves would score ln 145 ≈ 4.98, so the network learned clear move preferences. The value loss settled near 0.03.

Table 6: Training loss by iteration (final epoch)

Iteration	Training positions	Epochs	Total loss	Policy loss	Value loss	Time per epoch
3	60,677	4	4.83	4.78	0.052	≈15 s
4	121,733	4	4.25	4.19	0.053	≈33 s
5	202,916	5	3.66	3.62	0.041	≈54 s
6	281,526	5	3.36	3.33	0.035	≈73 s
7	330,296	6	3.39	3.35	0.033	≈90 s
8	382,755	6	3.34	3.31	0.031	≈118 s
9	415,990	8	3.27	3.24	0.031	≈136 s
10	446,449	8	3.29	3.26	0.034	≈78 s

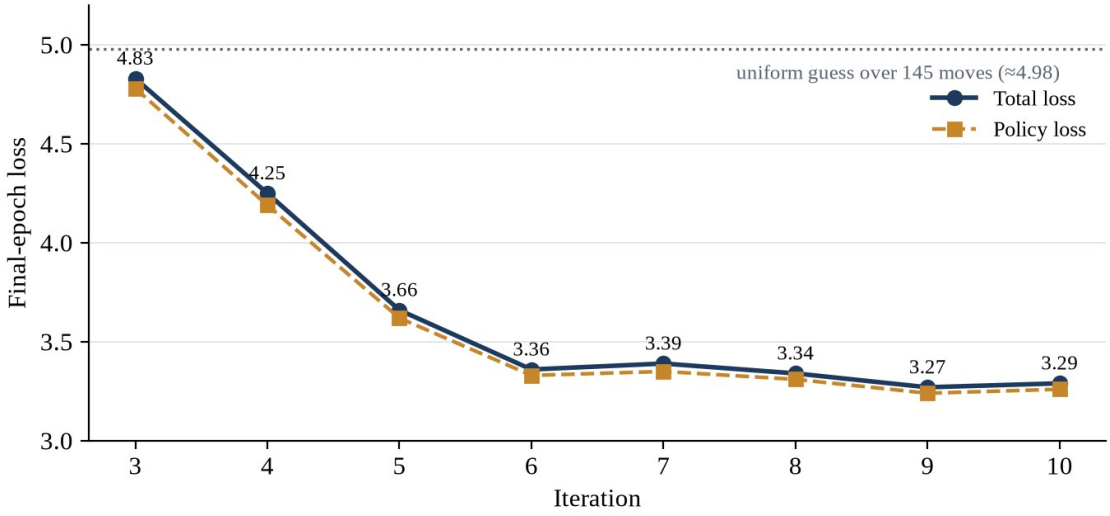


Figure 3: *Final-epoch training loss per iteration*

6.4 Cost of self-play

Self-play dominated the run time (Table 7). Iterations 3–10 played 5,700 games and produced about 1.23 million training positions in roughly 24.5 hours of self-play. Games grew from about 150 moves at stage 2 to about 225 moves later, as stronger models played more contested games.

Table 7: Self-play volume and time per iteration

Iteration	Games	Batches	New positions	Self-play time
3	400	16	60,677	32 min
4	400	16	61,056	33 min
5	600	24	141,860	106 min
6	600	24	139,666	103 min
7	850	34	190,630	227 min
8	850	34	192,125	277 min
9	1,000	40	223,865	400 min
10	1,000	40	222,656	289 min

Iterations 9 and 10 used the same settings and hardware but different operating systems. On Linux, a self-play batch of 25 games took 7.1 minutes on average against 10.0 minutes on Windows, and a training epoch took about 78 seconds against 136 seconds. We did not profile the cause; slower process creation on Windows is a likely factor. A GPU monitor showed 0% GPU use during evaluation, because MCTS evaluates one position at a time and runs on the CPU.

6.5 Discussion

The system learns. Three promotions show that the self-play loop produced stronger networks with no human data, as the paper claims. Iteration 9's result is statistically clear: if both models were equally strong, the standard error of a 40-game win rate would be $\sqrt{(0.25/40)} \approx 7.9$ percentage points, so 72.5% is about 2.8 standard errors above 50%.

Most rejections are ties, not regressions. Iterations 5, 6, 8 and 10 scored between 45% and 52.5%, within one standard error of an even match. A 40-game test cannot separate small improvements from noise. Even a candidate that is truly 5 points stronger would often fail the 55% bar. The paper's 400-game evaluations reduce this noise by a factor of about three.

Learning slows down. The loss curve flattens after iteration 6, at the same point where promotions become rare. Part of the remaining policy loss is unavoidable, because the targets π are themselves spread out: moves are sampled during the first 30 moves, and a 35–100 simulation search gives noisy visit counts. Further progress at this scale would need more games per iteration, a larger replay window or deeper search, all of which cost compute.

Limitations. We measured strength only relative to our own earlier models. We did not compare against an external program or compute an absolute rating, and the small 12×12 board and network mean our results show the method working, not superhuman play.

7. Conclusion

We reviewed *Mastering the game of Go without human knowledge* [1] and implemented its complete method: a Go environment, a dual-head residual network, PUCT-based MCTS with Dirichlet noise and temperature, parallel self-play, training with the AlphaGo Zero loss, and gated evaluation. Trained from scratch on a laptop through a four-stage curriculum, the system produced three promoted models. The best, iteration 9, won 72.5% of its games against its predecessor, and the policy loss fell from 4.78 to 3.24.

The project confirmed the paper's central claim at a small scale: search and learning can improve each other with no human knowledge beyond the rules. It also showed that the method is sensitive to implementation details. A single sign error in the MCTS backup made the program choose its opponent's best moves, and only careful evaluation revealed it. Finally, our results show the cost of the method: even on a 12×12 board, progress slows once the cheap gains are made, and measuring small improvements needs many evaluation games.

8. Future Work

- **Stronger evaluation:** play more games per evaluation, or keep an Elo ladder across all models, so that small improvements can be detected.
- **External baseline:** measure absolute strength against an established program such as GNU Go.
- **Batched GPU inference in MCTS:** evaluate many leaves at once, so the GPU is used during self-play and evaluation instead of sitting idle.
- **Closer to the paper:** add history planes to the input, a larger network, resignation to save compute, and a replay buffer measured in games rather than iterations.
- **AlphaZero-style training:** replace gating with continuous updates and compare the two approaches.
- **Larger boards:** move to 13×13 and 19×19 as compute allows.
- **Web application:** stream live self-play to the existing web interface over WebSockets.

9. References

[1] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel and D. Hassabis, “Mastering the game of Go without human knowledge,” *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.

[2] D. Silver et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[3] D. Silver et al., “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play,” *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[4] R. Coulom, “Efficient selectivity and backup operators in Monte-Carlo tree search,” in *Proc. 5th Int. Conf. Computers and Games*, LNCS 4630, Springer, 2006, pp. 72–83.

[5] L. Kocsis and C. Szepesvári, “Bandit based Monte-Carlo planning,” in *Proc. 17th European Conf. Machine Learning (ECML)*, LNCS 4212, Springer, 2006, pp. 282–293.

[6] C. D. Rosin, “Multi-armed bandits with episode context,” *Annals of Mathematics and Artificial Intelligence*, vol. 61, no. 3, pp. 203–230, 2011.

[7] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[8] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[9] J. Tromp and G. Farnebäck, “Combinatorics of Go,” in *Proc. 5th Int. Conf. Computers and Games*, LNCS 4630, Springer, 2006, pp. 84–99.

[10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[11] D. Foster, “AlphaGo Zero explained in one diagram,” *Applied Data Science* (Medium), 2017. [Online]. Available: https://medium.com/applied-data-science/alphago-zero-explained-in-one-diagram-365f5abf67e0

10. GitHub Repository and Reproducibility Details

Repository: https://github.com/chandrakanth-source/ALPHAGo-Zero (branch `shafreed`). Trained models are stored with Git LFS. A summary of every iteration is in `Final_Training_Report.md`.

Table 8: Repository structure

Path	Contents
environment/	Go rules: moves, captures, superko, passing, scoring
network/	GoNetwork (dual-head ResNet) and model loading
mcts/	MCTS, tree nodes, PUCT score, network evaluator
self_play/	Self-play games and parallel workers
training/	Trainer, dataset, checkpoint manager
evaluation/	Arena matches, parallel evaluation, promotion
train_batched.py	Batched, resumable curriculum trainer (stages 2–5)
models/	Trained models (Git LFS); latest_model.pt is the current best
tests/	Unit tests (pytest)
web/, index.html, app.js	Web interface for playing against the model

Environment. Python 3.14, PyTorch 2.11 with CUDA 12.8, NumPy; FastAPI and Uvicorn for the web interface. A CUDA GPU is optional for training but makes it much faster.

```
git clone https://github.com/chandrakanth-source/ALPHAGo-Zero.git
cd ALPHAGo-Zero && git checkout shafreed
git lfs pull                       # download the trained models
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt pytest httpx
pytest                             # run the test suite
```

Reproducing training. Each stage trains its two iterations in turn. Self-play is saved every 25 games, so the same command resumes an interrupted run.

```
python train_batched.py --stage 2     # iterations 3-4
python train_batched.py --stage 5     # iterations 9-10
python train_batched.py --all         # every stage in order
```

Expected cost. On our hardware, iterations 3–10 took about 24.5 hours of self-play plus training and evaluation. Self-play datasets (0.1–1.1 GB per iteration) are not committed, because they exceed GitHub's file size limit; they are regenerated by the commands above. Because self-play is random, a rerun produces different games, and win rates can vary by several points.